

Les sous-programmes

B.U.T. Informatique

Les sous-programmes

Sous-programme

Un sous-programme est un bloc d'instructions, en dehors du programme principal, que l'on appelle pour résoudre une tâche

- On décompose le problème et identifie des sous-problèmes
- On résout les sous-problèmes avec des sous programmes
- On n'a pas besoin de recopier le bloc d'instructions à chaque fois
- Les sous-programmes ne se résolvent que lorsque l'on les appelle



es sous-programmes

Un sous-programme possède :

- **Des entrées.** Ce sont toutes les informations dont a besoin le sous programme.
- **Une sortie.** C'est le résultat de la résolution du sous problème. On peut n'avoir aucune sortie.

Un sous-programme est spécifié par :

- Le **type du résultat** (de la sortie).
- Le **nom** du sous-programme. Même fonctionnement qu'avec le nom des variables
- Le **type, nom** et la **nature** des entrées. Le nom n'est valable qu'à l'intérieur de ce sous-programme

Procédures et fonctions

Il existe deux types de sous-programmes :

- Les **fonctions**, qui calculent et renvoient un résultat, à partir des **valeurs** en entrée
- Les **procédures** qui modifient des **variables** en entrée

Remarque

En C++, une procédure est une fonction qui renvoie un résultat de type vide (void).

Les Entrées

Il existe deux types de paramètre en entrée :

- Le passage de paramètre par **valeurs**. On affecte chaque valeur en paramètre à une nouvelle variable uniquement utilisé à l'intérieur du sous-programme. On parle aussi de **copie** car on copie une valeur dans une nouvelle variable.
- Le passage de paramètre par **variables**. On donne une variable au sous-programme qui va directement modifier cette variable et sa valeur. On parle de **référence**.

Remarque

Parfois la valeur donnée en entrée est de trop grande taille pour être recopiée à chaque appel. On utilise alors un passage en variable mais on précise que la variable est **constante**

Exemple – fonction

```
int f (int a, int b) {  
  
    return a*5+b*8+a*b;  
  
}
```

Si le type est différent de **void**, il s'agit d'une fonction et doit donc avoir un **return** (un résultat retourné). a et b sont ici des paramètre valeurs.

Exemple – procédure

```
void init (int & a) {  
  
    a = 0;  
  
}
```

void signifie qu'il s'agit d'une procédure. Le & indique que a est un paramètre passé en variable, on indique ainsi que sa valeur sera modifiée (sauf si *const*).

Appel d'une fonction

Après qu'une fonction a été défini, on l'appelle avec son nom et ses paramètres. On doit récupérer son résultat.

```
int f (int a, int b) {  
  
    return a*5+b*8+a*b;  
  
}
```

Est appelé ainsi:

```
int val1 = 2;  
int val2 = 3;  
  
int r = f(val1, val2);
```

L'ordre des paramètres doit être respecté.

Appel d'une procédure

Après qu'une procédure a été défini, on l'appelle avec son nom et ses paramètres.

```
void init (int & a) {  
  
    a = 0;  
  
}
```

Est appelé ainsi:

```
int val1 = 2;  
  
init(val1);
```

Exemple

```
void init (int & a) {  
    int val2 = 300;  
    a = 0;  
}  
int f (int a, int b) {  
    a = 20;  
    b = 0;  
    return 7;  
}
```

et le code principal:

```
int val1 = 2;  
int val2 = 3;  
  
init(val1);  
val2 = f(val1, val2);
```

val1 ? val2 ?

Mais le *main* du coup ?

Le *main* en C++, que l'on utilise depuis le début, est une fonction sans paramètre qui retourne un entier pour indiquer comment c'est passé le programme (0 si le programme c'est bien passé). Cette fonction est celle appelée lorsque l'on exécute le programme.

```
int main () {  
  
    return 0  
  
}
```

Résumé

Une procédure :

- Récupère des variables
- Modifie ces variables
- Peut être utilisée pour modifier des données
- Peut être utilisée pour saisir des données
- Peut être utilisée si un ensemble de résultat est produit

Une fonction :

- Récupère des valeurs
- Calcule un résultat à partir de ces valeurs
- Retourne le résultat
- Peut être utilisée dans une expression (ex : “ $y = f(x)$ ”)

Notions de bonne programmation

- **Encapsulation** : utiliser des sous-programmes pour servir d'interface pour manipuler des données et effectuer des routines
- **Modularité** : découper le code en petits sous-programmes. Pour “fabriquer” un programme, on peut simplement assembler des modules (sous-programmes)
- **Qualité** : Des sous-programmes courts, avec des noms clairs, et suffisamment nombreux permet de facilement comprendre et utiliser un code.

Récurtivité

Récurtivité

Un concept récursif est un concept qui est décrit à partir de lui même. Un sous-programme récursif est donc un programme qui s'appelle lui-même.

Récurtivité

Un concept récursif est un concept qui est décrit à partir de lui même. Un sous-programme récursif est donc un programme qui s'appelle lui-même.

- Sous-programmes demandant un peu plus d'expérience mais très propres à relire
- Récursif $\neq$ itératif

Récurtivité

Récurtivité

Un concept récursif est un concept qui est décrit à partir de lui même. Un sous-programme récursif est donc un programme qui s'appelle lui-même.

- Sous-programmes demandant un peu plus d'expérience mais très propres à relire
- Récursif $\neq$ itératif

Anecdote

GNU (emacs, gcc, gdb, bash, ...) signifie "GNU's Not UNIX"

Exemple

```
void f (int a) {  
    std::cout<<a<<std::endl;  
    if(a > 0) f(a-1);  
}
```

Un escalier de 50 marches c'est :

- Version itérative : la marche 1, la marche 2, ..., la marche 50
- Version récursive : la marche 50 et un escalier de 49 marches

- Toujours une conditionnelle (un *if*) avec deux branches:
 - l'une a un appel récursif (cas général)
 - l'autre n'a pas d'appel récursif (cas "terminaison")
- Globalement, une itérative peut se traduire en récursive et vice-versa
- Chaque appel utilise des données différentes

Exemple

```
void f (int a) {  
  
    // Un premier code ici  
    std::cout<<" Poupi_";  
  
    // La recursivite ou l'arr t  
    if(a > 0) f(a-1);  
    else std::cout<<" Pou_";  
  
}
```

Quel résultat pour f(2) ?